

Guía de Estudio - Tecnología Web II

HTML Semántico, Flexbox, CSS Grid, SASS/SCSS y UX/UI - Diseño Responsivo. Organizada igual que la chanchulla, con más profundidad y ejemplos de código.

1. HTML Semántico

La idea central: HTML no solo organiza el contenido de una página, también describe QUÉ SIGNIFICA cada parte. Un `<div>` le dice al navegador "aquí hay una caja", pero un `<nav>` le dice "aquí hay navegación". Esa diferencia importa para accesibilidad, SEO, y para que otro programador entienda el código sin leer las clases CSS.

- `<header>`: cabecera de la página o de una sección (logo, título, información introductoria, a veces el menú).
- `<nav>`: bloques importantes de navegación. No todos los enlaces de una web necesitan estar dentro de un nav.
- `<main>`: el contenido ÚNICO de la página. Regla clave: NO debería usarse para menú, logo, footer ni barra lateral.
- `<section>`: agrupación temática de contenido. Buena práctica: casi siempre lleva su propio título (h2/h3).
- `<article>`: contenido que tiene sentido POR SÍ MISMO (noticias, publicaciones, productos, entradas de blog, comentarios).
- `<aside>`: contenido complementario o relacionado (barra lateral, enlaces relacionados, publicidad).
- `<footer>`: pie de página o de sección (copyright, información institucional, contacto, redes sociales).

Ejemplo de estructura completa:

```
<header>...</header>
<nav>...</nav>
<main>
  <section>
    <article>...</article>
  </section>
  <aside>...</aside>
</main>
<footer>...</footer>
```

Error común: usar `<div class="header">` en vez de `<header>`. El navegador y las herramientas de accesibilidad no saben que ese div es una cabecera; con la etiqueta correcta, sí lo saben. Otro error: meter el logo o el menú dentro de `<main>`.

2. Flexbox: Fundamentos

Flexbox es un sistema de layout UNIDIMENSIONAL: organiza elementos en UNA fila o UNA columna (nunca las dos a la vez — para eso existe Grid). Se activa en el CONTENEDOR con `display: flex`, y automáticamente todos sus hijos directos se convierten en flex-items.

Ejemplo básico:

```
<div class="contenedor">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>

.contenedor { display: flex; background: lightblue; }
```

```
.item { background: coral; padding: 20px; margin: 10px; }
```

El concepto clave para entender TODO lo demás de Flexbox es el EJE:

- Eje PRINCIPAL: la dirección en la que fluyen los elementos (row = horizontal, column = vertical). Se controla con justify-content.
- Eje SECUNDARIO: perpendicular al principal. Se controla con align-items (para todos los items) o align-self (para uno solo).
- gap: espacio entre items SIN usar margin (evita tener que restar márgenes en los bordes).

Si cambiás flex-direction a column, los ejes se INVIERTEN: justify-content pasa a controlar el alto, y align-items el ancho.

3. justify-content

Distribuye y alinea los elementos dentro de un contenedor flex sobre el EJE PRINCIPAL.

- flex-start: los elementos se pegan al inicio (comportamiento por defecto).
- flex-end: los elementos se pegan al final.
- center: centra el grupo completo.
- space-between: distribuye el espacio ENTRE los elementos (los extremos quedan pegados a los bordes).
- space-around: espacio alrededor de cada elemento (el espacio en los bordes exteriores es la MITAD que el espacio entre elementos).
- space-evenly: el espacio es exactamente igual en todos los huecos, incluidos los bordes.

justify-content → eje principal. align-items → eje secundario. Confundir estos dos es el error más común al aprender Flexbox.

4. align-items

Alinea los elementos sobre el EJE SECUNDARIO del contenedor flex, afectando a TODOS los items a la vez.

- stretch: valor por defecto. Los items se estiran para ocupar todo el alto disponible (solo si no tienen una altura fija propia).
- flex-start: todos se van al inicio del eje secundario (arriba, si es fila).
- flex-end: todos se van al final (abajo, si es fila).
- center: centrados en el eje secundario.
- baseline: alinea según la línea base del TEXTO de cada item, no según la caja completa — útil cuando hay textos de distinto tamaño de fuente.

5. flex-direction, order y align-self

flex-direction — define la dirección del eje principal:

- row: horizontal (por defecto).
- row-reverse: horizontal invertido.
- column: vertical.
- column-reverse: vertical invertido.

order — cambia el orden visual sin tocar el HTML:

- El valor por defecto de todos los items es 0.

- Menor número aparece primero. Acepta negativos (ej: order: -1 pone ese item antes que todos los que tienen 0).
- Con números altos (ej: 999) se puede mandar un item al final sin importar los demás.

align-self — como align-items pero para UN item puntual:

- align-items afecta a TODOS los items del contenedor.
- align-self afecta solo al item donde se escribe, y sobrescribe la regla general.
- Valores: auto (hereda), stretch, flex-start, flex-end, center, baseline.

6. flex-wrap y align-content

flex-wrap — controla si los items saltan de línea cuando no caben:

- nowrap: valor por defecto, NO permite salto (los items se achican o desbordan).
- wrap: permite que los items salten a una nueva línea.
- wrap-reverse: permite el salto pero en orden invertido.

align-content — distribuye FILAS completas dentro del contenedor:

- Solo tiene efecto si hay flex-wrap: wrap Y más de una línea de elementos. Con una sola fila, NO hace nada.
- Valores: stretch, flex-start, flex-end, center, space-between, space-around, space-evenly (mismo significado que en justify-content, pero aplicado a filas enteras).

7. CSS Grid: Fundamentos

A diferencia de Flexbox, Grid es un sistema de layout BIDIMENSIONAL: controla filas Y columnas al mismo tiempo. Se activa con display: grid en el contenedor; los hijos directos se convierten en grid-items.

Ejemplo básico (3 columnas iguales):

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

- grid-template-columns / grid-template-rows: definen cuántas columnas o filas hay y su tamaño. 1fr = una fracción del espacio disponible.
- gap: separación entre celdas (mismo concepto que en flexbox).
- grid-column / grid-row: ubican un item usando LÍNEAS (no números de columna). grid-column: 1 / 3 significa "desde la línea 1 hasta la línea 3" (ocupa 2 columnas).

Celdas grandes (un item que ocupa varias celdas):

```
.big {
  background: tomato;
  grid-column: 2 / 4; /* columnas 2 y 3 */
  grid-row: 2 / 4; /* filas 2 y 3 */
}
```

Para que un grid quede completamente lleno, hay que poner tantos .item como celdas totales (ej: una grilla 3x3 necesita 9 divs). Si un item usa grid-column/row para ocupar más de una celda, hay que restar esas celdas del conteo total.

8. CSS Grid: Avanzado

Columnas automáticas responsivas (sin escribir media queries):

```
.contenedor {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
}
```

Esto crea tantas columnas como quepan en el contenedor, cada una de mínimo 200px, y se expanden para llenar el espacio disponible. Es la forma "inteligente" de hacer columnas responsivas.

grid-template-areas — mapa nombrado del layout:

```
.layout {
  display: grid;
  grid-template-columns: 300px 1fr;
  grid-template-rows: 100px 1fr 60px;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer { grid-area: footer; }
```

- Cada línea entre comillas representa una fila del layout.
- Cada palabra dentro de las comillas es el nombre de un área en esa columna.
- Regla obligatoria: un área nombrada debe formar un RECTÁNGULO perfecto (no se puede hacer una forma de L, por ejemplo).
- height: 100vh se usa seguido junto con esto para que el layout ocupe toda la altura de la ventana del navegador ("viewport height").

Cuándo usar Flexbox y cuándo Grid: Flexbox para una fila o columna de elementos (un menú, una fila de botones, tarjetas de igual tamaño). Grid para la estructura GENERAL de la página o cualquier cosa que necesite filas Y columnas relacionadas entre sí (como el layout header/sidebar/content/footer de arriba).

9. SASS/SCSS: Conceptos e Instalación

SASS (Syntactically Awesome Style Sheets) es un PREPROCESADOR de CSS: se escribe en un archivo .scss con superpoderes (variables, anidación, mixins) y luego se COMPILA a un archivo .css normal, que es el que el navegador realmente entiende. SASS es la tecnología; SCSS es la sintaxis moderna (parecida a CSS, con llaves y punto y coma).

Pasos de instalación y uso:

1. Verificar/installar Node.js: node -v y npm -v en la terminal.
2. Instalar SASS globalmente: npm install -g sass — verificar con sass --version.
3. Compilar una vez: sass scss/estilo.scss css/estilo.css
4. Compilar automáticamente cada vez que se guarda: sass --watch scss:css

El archivo .css.map que se genera junto al .css compilado es solo un "traductor" que le dice al navegador de qué línea del .scss vino cada estilo (útil para depurar en las herramientas de desarrollador). No es obligatorio subirlo a producción.

Variables y anidación:

```
$color-principal: #3498db;

nav {
  background: black;
  ul { display: flex; }
  a { color: $color-principal; }
}
```

- Variables: guardan un valor para reutilizarlo en todo el archivo. Si después cambia el color de marca, se edita en un solo lugar.
- Anidación: escribir un selector DENTRO de otro para reflejar la jerarquía del HTML (nav { a {...} } equivale a "nav a" en CSS normal). Evita repetir el selector padre.

10. SASS/SCSS: Mixins y Aplicación en Maquetación

Un mixin es un bloque de estilos reutilizable, opcionalmente con parámetros. Se define con @mixin y se usa con @include.

```
@mixin flex($justify: center, $align: center, $dir: row) {
  display: flex;
  justify-content: $justify;
  align-items: $align;
  flex-direction: $dir;
}

.header { @include flex; }
.menu { @include flex(flex-start, flex-start, column); }
```

- Los parámetros pueden tener un valor POR DEFECTO (ej: \$justify: center), así se puede usar @include flex; sin escribir nada, o @include flex(space-between); para cambiar solo lo necesario.
- El símbolo & dentro de un bloque anidado representa al selector PADRE. Es clave para el patrón BEM de modificadores: .btn { &--primary { background: blue; } } compila a .btn--primary.

En una maquetación típica (header + nav + main + aside + footer), se suele definir un mixin de flex reutilizable y aplicarlo en varias secciones distintas, cambiando solo la dirección o la alineación según haga falta — así se evita repetir display:flex una y otra vez.

11. UX/UI y Diseño Responsivo

UI (User Interface) es la parte VISIBLE de un sitio: colores, tipografía, botones, espaciado, íconos. UX (User Experience) es la experiencia COMPLETA de usarlo: si es fácil de navegar, si se entiende el contenido, si funciona bien en el celular, si carga rápido. Un sitio puede tener buena UI (verse bonito) y mala UX (ser difícil de usar) — son cosas relacionadas pero distintas.

Diseño responsivo NO es hacer 3 sitios distintos (uno para móvil, otro para tablet, otro para desktop). Es hacer UN sitio flexible que se adapta usando Flexbox, Grid y media queries.

Ejemplo de media query (mobile-first):

```
/* Base: pensado para móvil, sin media query */
.nav { display: flex; flex-direction: column; }
.cards { display: grid; grid-template-columns: 1fr; }

/* Cuando la pantalla mida 768px o más */
@media (min-width: 768px) {
  .nav { flex-direction: row; }
  .cards { grid-template-columns: 1fr 1fr; }
}
```

Comparación clave: un sitio "versión 1" (desktop-first, con columnas fijas pensadas solo para pantalla grande) se ROMPE en móvil — el menú y las tarjetas se aprietan o desbordan. Un sitio "versión 2" (mobile-first) arranca con nav vertical y una sola columna de tarjetas SIN media query, y recién agrega columnas/filas horizontales a partir de los breakpoints hacia arriba.

12. px vs rem y Breakpoints

- px (píxeles): unidad FIJA, no cambia sin importar la configuración del navegador o del elemento raíz. Buena para bordes, sombras y detalles pequeños.
- rem (root em): unidad RELATIVA al font-size del elemento <html> (la raíz). Si `html { font-size: 16px }`, entonces `1rem = 16px` y `2rem = 32px`. Buena para tipografía y espaciados, porque si alguien cambia el tamaño base, todo el sitio escala junto.
- em: relativo al font-size del elemento PADRE (no de html) — por eso se usa menos, puede acumularse de forma impredecible si hay muchos niveles anidados.

Tabla de uso recomendado:

- Bordes y sombras → px (no necesitan escalar).
- Tamaño de fuente y espaciado → rem (para que todo el sitio crezca proporcional).
- Layout fijo pequeño → px.

Breakpoints comunes (rangos aproximados de dispositivo):

- Móvil: 0 - 767px (base, SIN media query).
- Tablet: ≥768px → @media (min-width: 768px)
- Laptop: ≥1024px → @media (min-width: 1024px)
- Desktop: ≥1200px → @media (min-width: 1200px)
- Pantalla grande: ≥1400px → @media (min-width: 1400px)